

Post-Hoc Dynamic Tanh: LayerNorm Replacement in Pretrained Transformer Encoders for Classification and Semantic Segmentation

Kyle Luo

Massachusetts Institute of Technology

zhizhen@mit.edu

Abstract

Dynamic Tanh (DyT) is a learnable element-wise replacement for transformer normalization layers that has been validated when trained from scratch. We study a different regime: replacing LayerNorm with DyT inside a LayerNorm-pretrained vision encoder, then fine-tuning on a downstream task. We test this post-hoc replacement on two computer vision tasks: image classification (ViT-B/16 fine-tuned on CIFAR-100) and semantic segmentation (SegFormer-B0 fine-tuned on a 2000-image ADE20K subset). We compare the paper-default DyT initialization ($\gamma=1$, $\beta=0$, $\alpha_0=0.5$) with an affine-preserving adaptation that copies the pretrained LayerNorm γ, β into DyT at replacement time. On CIFAR-100, affine preservation improves post-hoc DyT from 4.50% to 48.32% top-1, still well below the 92.91% LayerNorm baseline. On the ADE20K subset, both DyT variants fail near chance (0.0099 and 0.0138 mIoU, $3-4\times$ the ≈ 0.0033 chance baseline), while LayerNorm reaches 0.0765, roughly $23\times$ chance. Diagnostics identify two distinct failure modes: tanh saturation in ViT default that affine-preserving initialization removes, and a loss of per-token mean-and-variance normalization in SegFormer that no tested initialization recovers. Full post-hoc DyT replacement is therefore not a safe drop-in substitute for pretrained vision encoders.

1. Introduction

Transformer-based models have become standard components of modern computer vision systems, including image classification, semantic segmentation, and other dense prediction tasks [3, 18]. These models commonly inherit the same transformer block structure: attention, feed-forward layers, residual connections, and normalization [16]. LayerNorm is especially central to this design, and it is often treated as a default part of the transformer stack rather than as a component whose necessity should be tested for each visual task.

Recent work challenges this assumption. Zhu et al. propose Dynamic Tanh (DyT), a learnable element-wise transformation that replaces transformer normalization layers without computing sample-wise means or variances [22]. DyT is introduced as a drop-in replacement for LayerNorm or RMSNorm, and the authors validate it across settings ranging from recognition to generation, supervised to self-supervised learning, and computer vision to language models [22]. This suggests that some of LayerNorm’s practical role may be reproduced by a simpler bounded nonlinearity.

The DyT paper reports broad transformer results, but in every reported experiment DyT is in the network from the start of training [22]. It does not test the post-hoc workflow common in applied computer vision: start from a public LayerNorm-pretrained backbone, replace its LayerNorm modules, and fine-tune on a downstream task. This kind of normalization surgery is plausible. Heimersheim shows that LayerNorm can be removed from GPT-2 by fine-tuning [4]. No analogous test exists for DyT in a pretrained vision transformer.

This regime matters because most computer vision applications do not train from scratch. The dominant practice is to download a public pretrained checkpoint and fine-tune it on a downstream task [6, 7]. A normalization replacement that only works under from-scratch training therefore reaches a much narrower audience than one that survives this workflow. Additionally, semantic segmentation remains untested by the DyT paper. Semantic segmentation is a dense prediction task that assigns a class label to each pixel without distinguishing separate instances of the same class [10, 15]. Ranftl et al. observe that feature resolution and granularity, which can be discarded in classification, are critical for dense prediction [14]. Evaluating DyT in both classification and segmentation therefore widens the test beyond image-level recognition.

This work evaluates whether DyT can replace LayerNorm in this post-hoc fine-tuning regime, in two pretrained vision encoders: ViT-B/16 [3] fine-tuned on CIFAR-100 [8], and SegFormer-B0 [18] fine-tuned on an ADE20K subset [21]. In each setting we compare three variants: the

LayerNorm baseline, DyT default following Algorithm 1 of Zhu et al. [22], and a single proposed adaptation that copies the pretrained LayerNorm affine parameters into DyT’s γ and β at the moment of replacement. Evaluation pairs the downstream metric (top-1 accuracy or mIoU) with convergence curves, per-layer trained α , and DyT activation saturation.

This work makes three main contributions. First, it presents the first evaluation of post-hoc DyT replacement in pretrained vision encoders, covering both image classification (ViT-B/16 on CIFAR-100) and semantic segmentation (SegFormer-B0 on an ADE20K subset). Second, it introduces an affine-preserving DyT initialization that copies the pretrained LayerNorm γ, β into DyT, which recovers about half of the post-hoc classification gap relative to the paper-default initialization but does not improve segmentation. Third, it identifies two distinct failure modes through per-layer α and saturation diagnostics: tanh saturation in ViT default that affine-preserving initialization removes, and a loss of per-token mean-and-variance normalization in SegFormer that no tested initialization recovers.

2. Related Work

2.1. Transformer Backbones for Vision

Vision Transformers showed that pure transformer encoders can match convolutional networks on image classification when pretrained at sufficient scale [3]. Subsequent work adapted transformer architectures for pixel-level prediction: SegFormer pairs a hierarchical transformer encoder with a lightweight MLP decoder for efficient semantic segmentation [18], and Mask2Former uses masked attention as a unified architecture for semantic, instance, and panoptic segmentation [2]. All of these models retain LayerNorm as a default in their transformer components. This work adopts ViT-B/16 and SegFormer-B0 as testbeds; neither has been evaluated with DyT.

2.2. Normalization in Transformers

LayerNorm normalizes hidden features within each example, removing the dependence on batch statistics used by earlier methods such as Batch Normalization [1, 5]. It became a standard component of transformer blocks, where it is used with residual connections, attention layers, and feed-forward sublayers [16]. Later work shows that the details of normalization matter. RMSNorm removes mean-centering while retaining re-scaling, suggesting that not all parts of LayerNorm are equally necessary [20]. Analyses of Pre-LN and Post-LN transformers show that normalization placement affects gradient behavior and training stability [19]. These studies treat normalization as a design choice rather than a fixed component. Each modifies normalization rather than removing or replacing it outright.

2.3. Dynamic Tanh and Normalization-Free Transformers

Dynamic Tanh replaces transformer normalization with a learnable bounded nonlinearity [22]. Given an input x , DyT applies

$$\text{DyT}(x) = \gamma \odot \tanh(\alpha x) + \beta, \quad (1)$$

where α is a learnable scalar and γ, β are learnable affine parameters. Unlike LayerNorm, DyT does not compute a sample-wise mean or variance. Instead, it uses a bounded element-wise nonlinearity followed by learned scaling and shifting.

The motivation for DyT is that trained LayerNorm modules in transformers often exhibit tanh-like input-output behavior [22]. Zhu et al. show that replacing normalization layers with DyT can match or exceed normalized counterparts across several transformer settings, challenging the assumption that normalization is indispensable. Heimersheim shows that LayerNorm can be removed from a pretrained GPT-2 by fine-tuning, providing one instance of post-pretraining normalization surgery in the language-model setting [4]. This work adopts the DyT formulation directly to test the analogous question in pretrained vision transformer encoders.

3. Methods

The methodology is a controlled post-hoc intervention on pretrained models. Rather than training a DyT transformer from scratch, we start from a LayerNorm-pretrained checkpoint, replace the relevant LayerNorm modules, and fine-tune the modified model on the same downstream data as the unmodified baseline. This setup directly tests the practitioner-facing question from the introduction: whether DyT can be inserted into a model that was not originally trained with DyT. The comparison is intentionally narrow. Within each task, the LayerNorm baseline and both DyT variants share the same data, optimizer, training length, and evaluation procedure, so performance differences can be attributed primarily to the normalization replacement and DyT initialization. Figure 1 summarizes the workflow.

3.1. Post-hoc replacement and backbones

Zhu et al. validate DyT only when it is present from the start of training or pretraining; they do not swap DyT into an existing LayerNorm checkpoint [22]. We isolate this post-hoc setting: each variant starts from a public pretrained checkpoint, performs the LayerNorm-to-DyT replacement before the first optimizer step, and then fine-tunes all parameters on the downstream task. This tests whether downstream fine-tuning can absorb the changed normalization map while the rest of the pretrained weights start unchanged.

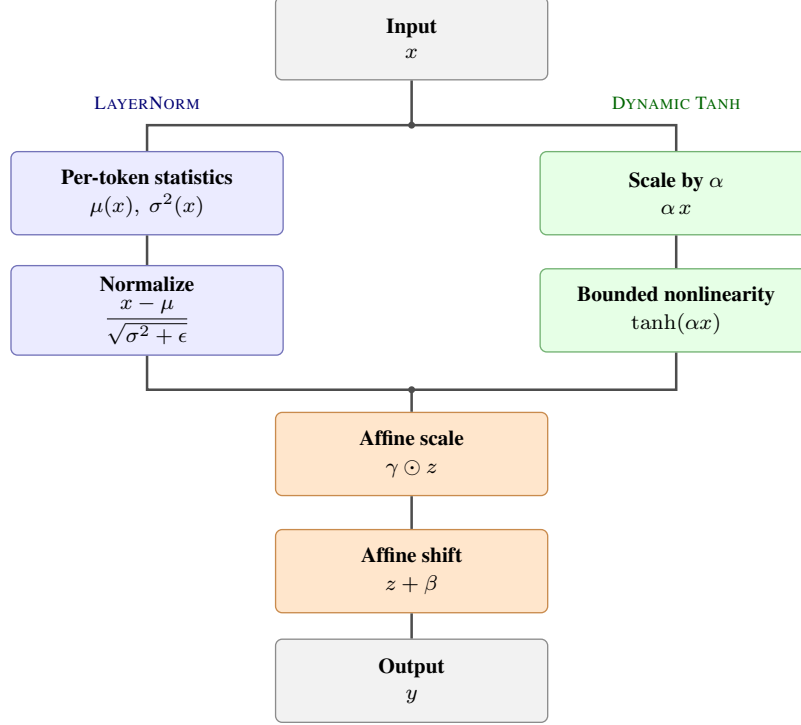


Figure 1. LayerNorm and Dynamic Tanh as computation graphs. Both share an identical input, per-channel affine map ($\gamma \odot z + \beta$), and output, and differ only in the middle two stages: LayerNorm computes per-token mean and variance and normalizes (blue), while DyT element-wise scales by α and applies a bounded tanh (green). The orange affine is structurally shared. We test two initializations of DyT’s γ, β at the moment of replacement (Sec. 3.3).

We use ViT-B/16 [3] for classification, fine-tuned on CIFAR-100 [8], loaded from google/vit-base-patch16-224 through HuggingFace Transformers [17]. CIFAR-100 is a feasible classification anchor under our single-GPU class-project budget; full ImageNet-1K from-scratch training, which Zhu et al. report in their Table 1, is infeasible in this regime, and an ImageNet-100 subset would add substantial setup overhead for marginal scientific gain over CIFAR-100 in a within-project comparative study. ViT-B/16 itself is the same architecture Zhu et al. report in their Table 1, which keeps the model axis aligned with the original DyT evaluation. We do not test DeiT (same architecture as ViT, different pretraining recipe) or ConvNeXt (CNN, out of scope for LN-in-transformer replacement).

For segmentation we use SegFormer-B0 [18] with the MiT-B0 encoder, loaded from nvidia/mit-b0, fine-tuned on a fixed-seed ADE20K [21] subset that preserves ADE20K’s 150-class semantic diversity at one-tenth the training data. Full ADE20K with the official 160k-iteration SegFormer recipe is outside budget [12, 18]. PASCAL VOC trades semantic diversity for speed, and Cityscapes’ high-resolution urban scenes amplify compute cost without adding scope to a normalization study.

The replacement walks `model.named_modules()` and substitutes every `nn.LayerNorm` under a task-specific prefix: `vit.` for ViT and `segformer.encoder.` for SegFormer. The ViT prefix matches 25 LayerNorms: 24 inside the transformer blocks plus the final pre-classifier `vit.layernorm`. The SegFormer prefix matches 30 LayerNorms across the MiT-B0 encoder: patch embeddings, transformer blocks, stage outputs, and spatial-reduction modules. We do not exclude any LayerNorm by choice: the ViT classifier head is a single `Linear` with no normalization, and SegFormer’s all-MLP decode head uses BatchNorm rather than LayerNorm, so the encoder prefix already covers every LayerNorm in either model [18].

3.2. LayerNorm and Dynamic Tanh

For an input feature vector $x \in \mathbb{R}^d$, LayerNorm normalizes along the channel dimension using the sample’s mean and variance:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} + \beta. \quad (2)$$

The constant ϵ prevents division by zero when a layer’s input has near-zero variance. We do not set ϵ ourselves.

Each checkpoint ships its own value, and we inherit it: 10^{-12} for `google/vit-base-patch16-224`, 10^{-6} for `nvidia/mit-b0`. DyT (Eq. (1)) replaces this sample-wise normalization, with the per-channel γ, β taking the same shape as in Eq. (2) and no ϵ because no division occurs.

3.3. Two initializations: DyT default and DyT affine

Algorithm 1 of Zhu et al. initializes DyT with $\gamma = \mathbf{1}$, $\beta = \mathbf{0}$, and $\alpha = 0.5$ [22]. In a post-hoc checkpoint, downstream weights were trained against the LayerNorm affine output $\gamma_{\text{LN}}, \beta_{\text{LN}}$. Resetting those parameters changes both the normalization map and its learned affine range.

We therefore study two DyT initializations in the post-hoc regime:

- DyT default: $\gamma = \mathbf{1}$, $\beta = \mathbf{0}$, $\alpha = 0.5$, exactly as Algorithm 1. Zhu et al. report from-scratch training to be insensitive to α_0 in $[0.5, 1.2]$ (their Figure 9), so we hold $\alpha_0 = 0.5$ in this paper rather than sweep it.
- DyT affine: $\gamma \leftarrow \gamma_{\text{LN}}$, $\beta \leftarrow \beta_{\text{LN}}$, $\alpha = 0.5$. The LayerNorm weight and bias tensors are copied into DyT’s γ and β ; α has no LayerNorm analog and stays at the default.

These two DyT variants are compared against an unmodified LayerNorm baseline on each of the two tasks, for six runs total.

4. Experimental Setup

4.1. Datasets

Classification. CIFAR-100 [8] is loaded from `uoft-cs/cifar100` through HuggingFace Datasets [9]. **Segmentation.** ADE20K [21] is loaded from the `merge/scene_parse_150` HuggingFace mirror through HuggingFace Datasets [9]. The subset uses 2000 train images and 500 validation images, drawn at fixed seed; all segmentation variants share the same indices.

4.2. Training setup

The training setup is designed for controlled within-task comparison rather than for a maximally tuned benchmark. Each task is run in three configurations: a LayerNorm baseline, DyT default, and DyT affine. Within a task, these configurations use the same seed (42), data subset, optimizer, learning-rate decay rule, augmentation policy, and evaluation cadence. The intended differences are only the normalization module and, for DyT, the initialization.

The per-task fine-tuning recipes are listed in Tab. 1. We use one learning rate per task and do not tune it separately for each configuration, so the comparison isolates the normalization replacement rather than per-configuration hyperparameter search. These are shortened fine-tuning runs

for within-task comparison, not reproductions of the from-scratch ImageNet experiments in the original DyT evaluation [22].

Table 1. Training setup per task. AdamW [11], cross-entropy loss, and mixed precision are shared across both. Image normalization uses each checkpoint’s HuggingFace preprocessor configuration.

	Classification	Segmentation
Iterations	3000	5000
Batch size	64	4
Crop	224	512×512
Learning rate	5×10^{-5}	6×10^{-5}
Weight decay	0.01	0.01
Warmup iters	200	200
LR decay	cosine	poly ($p=1.0$)
Eval cadence	every 500	every 500

Train loaders shuffle to avoid a fixed data order and drop the final incomplete batch so every optimizer step has the same batch size. Validation loaders do not shuffle because the reported metrics are order-invariant. We use standard augmentations for each task: random resized crops and horizontal flips for CIFAR-100, and random scale/crop, horizontal flips, and color jitter for ADE20K. At validation time, classification uses a fixed center crop, while segmentation uses a fixed resize/crop and upsamples logits to label resolution before scoring. Pixels labeled 255 are ignored.

All runs use Colab Pro on a single NVIDIA A100-SXM4-80GB with PyTorch [13] automatic mixed precision.

To support reproducibility, the accompanying notebook records the checkpoint identifiers, replacement prefixes, dataset subset sizes, fixed seed, and run configurations used to generate the tables and figures. Each completed run also saves its configuration, training log, and summary metrics.¹ This is important for this project because the main claim is a negative comparison: without the shared run settings and logs, a DyT failure could be mistaken for an implementation or tuning artifact.

4.3. Evaluation metrics and diagnostics

For classification, we use top-1 accuracy for single-label image classification [15]. Let y_i be the ground-truth class for validation image i and let $s_{i,c}$ be the model score for class c . The reported percentage is $100 \times \text{Top-1}$, where

$$\text{Top-1} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[\arg \max_c s_{i,c} = y_i \right]. \quad (3)$$

For segmentation, we use the standard mean-IU/mIoU metric for semantic segmentation [10]. For class c , with true

¹Code and run logs: <https://github.com/zhizhen-kyle-luo/6s058-final-project>

positives TP_c , false positives FP_c , and false negatives FN_c accumulated over the validation set,

$$\begin{aligned} \text{IoU}_c &= \frac{TP_c}{TP_c + FP_c + FN_c}, \\ \text{mIoU} &= \frac{1}{C} \sum_{c=1}^C \text{IoU}_c. \end{aligned} \quad (4)$$

Here $C = 150$ for ADE20K, and pixels labeled 255, which mark ADE20K’s unannotated regions, are excluded from both the loss and the IoU counts. We compute the counts with a streaming confusion matrix. Training loss is logged every 50 iterations and validation metrics every 500 iterations. For DyT runs we also record:

- the final learned α scalar for each DyT layer;
- the fraction of recorded activations satisfying $|\tanh(\alpha x)| > 0.95$, averaged per layer.

5. Results

The results answer three questions. First, full post-hoc DyT replacement does not preserve the LayerNorm baseline. Second, copying the pretrained LayerNorm affine parameters helps classification but not segmentation; the default-versus-affine comparison is an ablation of whether preserving these parameters matters. Third, the diagnostics do not point to a single failure mode across tasks. All numbers are from a single seed per configuration; the LayerNorm-versus-DyT gaps dwarf plausible seed variance, but the small within-DyT differences on segmentation should be read with this caveat.

5.1. Classification

Table 2. CIFAR-100 fine-tune results for ViT-B/16. All variants share the same protocol; only the normalization module and DyT initialization differ.

Variant	Top-1 (%) $\uparrow$
LayerNorm baseline	92.91
DyT default ($\gamma=\mathbf{1}, \beta=\mathbf{0}, \alpha_0=0.5$)	4.50
DyT affine ($\gamma \leftarrow \gamma_{\text{LN}}, \beta \leftarrow \beta_{\text{LN}}, \alpha_0=0.5$)	48.32

Table 2 shows a split result. DyT default nearly fails on CIFAR-100, while DyT affine becomes a nontrivial classifier. This suggests that the affine reset is an important part of the ViT failure mode. However, DyT affine remains far below the LayerNorm baseline, so preserving γ and β alone does not make full post-hoc DyT replacement viable.

5.2. Segmentation

Table 3 shows a different pattern from classification. As a reference, under uniform random class predictions over

Table 3. ADE20K-subset fine-tune results for SegFormer-B0. All variants use the same 2000-image training subset, 500-image validation subset, and training recipe; the mIoU values are not intended as full ADE20K benchmark numbers.

Variant	mIoU $\uparrow$
LayerNorm baseline	0.0765
DyT default ($\gamma=\mathbf{1}, \beta=\mathbf{0}, \alpha_0=0.5$)	0.0138
DyT affine ($\gamma \leftarrow \gamma_{\text{LN}}, \beta \leftarrow \beta_{\text{LN}}, \alpha_0=0.5$)	0.0099

$C = 150$ ADE20K classes with balanced class proportions $p_c \approx 1/C$, per-class IoU is $p_c/(1 + p_c(C - 1)) \approx 1/(2C)$, so chance mIoU is roughly $1/(2C) \approx 0.0033$. Both DyT variants sit within a small constant of this floor: DyT default at 0.0138 and DyT affine at 0.0099 are $\approx 3\text{--}4\times$ chance, while the LayerNorm baseline at 0.0765 is $\approx 23\times$ chance. Copying the pretrained affine parameters does not improve segmentation, so the classification recovery does not transfer to dense prediction. This suggests that the SegFormer failure is not explained by the same affine-reset effect observed in ViT. The absolute mIoU values should be read cautiously because the ADE20K split is small and the decode head is trained from scratch, but the shared protocol still shows a clear gap between LayerNorm and both DyT variants.

5.3. Optimization curves

Figure 2 shows that the endpoint gaps reflect different training dynamics. In classification, the LayerNorm model completes most of its fine-tuning by the first validation checkpoint, while DyT default stays near the random cross-entropy scale for CIFAR-100 ($\log 100 \approx 4.61$) and barely improves in top-1. Affine preservation changes the trajectory: validation accuracy rises throughout training, indicating that copying γ and β makes the post-hoc ViT optimization problem less stuck, although still far from the LayerNorm baseline.

Segmentation behaves differently. The LayerNorm run improves steadily in mIoU over the full schedule, whereas both DyT variants make only small early gains and remain near chance. Their training losses still move and fluctuate, so the failure is not a clean numerical divergence. Instead, the models appear able to reduce pixelwise cross-entropy somewhat without learning segmentations that improve class-level overlap. This mismatch between loss and mIoU is consistent with a dense-prediction failure rather than a simple need for more iterations.

5.4. Per-layer α and DyT saturation

Figure 3 shows an asymmetry between the two ViT DyT variants. DyT default α stays nearly frozen at the initialization $\alpha_0 = 0.5$ across all 25 layers, whereas DyT affine α



Figure 2. Training loss and validation metric versus iteration. Top row: classification loss and top-1. Bottom row: segmentation loss and mIoU.

drifts above 0.5 in the early and middle blocks and spikes at the final pre-classifier layer. DyT default therefore did not learn to adjust α , consistent with its near-zero classification progress; the affine variant trained α in addition to γ and β . Segmentation α drifts in both variants (per-run means 0.520–0.531), so the segmentation failure is not a case of α failing to update.

Saturation in Figure 4 explains the ViT contrast. DyT default saturation grows with depth and peaks near 0.96 at the final pre-classifier layer (mean 0.76, 21 of 25 layers above 0.5), while DyT affine is uniformly low (mean 0.17, only 4 of 25 layers above 0.5). Copying γ, β therefore keeps every ViT layer in $\tanh$ ’s near-linear regime, consistent with the classification recovery. SegFormer behaves differently. Both DyT variants run at very low saturation (means 0.007 and 0.060), so $|\alpha x|$ is small for nearly all activations; first-order Taylor expansion of $\tanh$ around 0 gives $\tanh(\alpha x) \approx \alpha x$, so DyT collapses to a per-channel affine map:

$$\text{DyT}(x) \approx \gamma \odot (\alpha x) + \beta = (\alpha \gamma) \odot x + \beta. \quad (5)$$

That this affine-only configuration still fails to learn segmentation rules out $\tanh$ clipping as the SegFormer failure mode. The remaining structural difference between DyT-

affine and LayerNorm in this regime is per-sample mean-centering and variance-scaling.

6. Discussion

The post-hoc and from-scratch regimes are genuinely different problems. Downstream layers in a pretrained transformer were optimized against LayerNorm’s per-token mean-centered and variance-scaled features; swapping LayerNorm for DyT replaces this per-token adjustment with a fixed channel-wise affine, so the downstream weights begin fine-tuning from inputs whose first and second moments no longer match what they were trained on, and the present results show that the budgets we tested do not close that mismatch. The classification-versus-segmentation asymmetry is consistent with Ranftl et al.’s observation that dense prediction depends on feature granularity that classification can discard [14]. Copying γ, β is enough to recover a useful image-level representation but not enough to recover the spatially detailed features SegFormer needs. Concretely, LayerNorm’s per-token mean-centering and variance-scaling adapts to each token’s mean and variance over the channel dimension, while DyT’s channel-wise γ, β apply the same transform across all tokens; the per-token adaptation is what Eq. (5) loses.

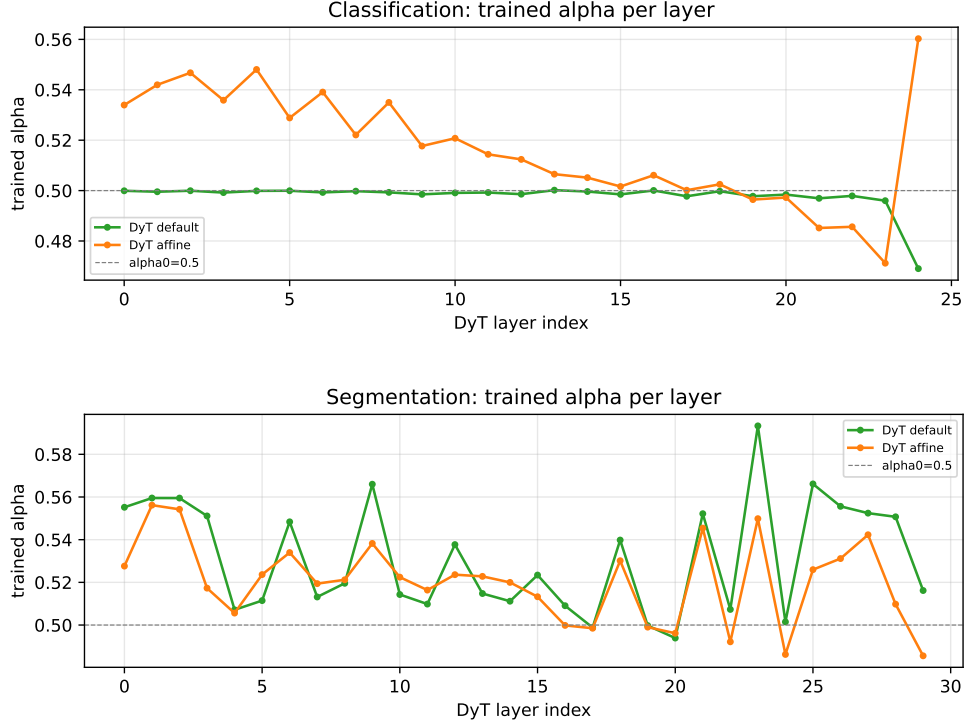


Figure 3. Per-layer trained α for both DyT variants. Top: classification (ViT-B/16, 25 DyT modules). Bottom: segmentation (SegFormer-B0, 30 DyT modules). The horizontal line at $\alpha_0 = 0.5$ marks initialization.

The above conclusions are drawn within the ViT-B/16 and SegFormer-B0 scale. Larger encoders may exhibit different per-token feature scales and different depth-dependent saturation profiles than Figure 4 shows, so the pattern reported here should not be extrapolated upward without retesting. We also do not claim that DyT is intrinsically weak: the original paper shows strong from-scratch results across many settings. Within the post-hoc setting, our results identify a baseline that several follow-up directions can be measured against. Compute scaling is the most direct: the original DyT evaluation runs roughly an order of magnitude longer than our fine-tunes, so a longer schedule, especially on segmentation, would test whether the gap we report is fundamental or budget-bounded. Within the same budget, several adaptations are natural: site-selective replacement that excludes the worst sites (the final pre-classifier LN, where saturation peaks); activation-matching initialization that calibrates α, γ, β to the pretrained LayerNorm’s input-output statistics on the first batches of fine-tuning; and DyT variants that restore some form of per-token scaling, directly addressing the candidate explanation above. Broader testing on Swin, MAE-pretrained ViT, and dense-prediction backbones beyond MiT-B0 would clarify how general the classification-versus-segmentation asymmetry is.

6.1. Ethical considerations

This work introduces no dataset, no human-subject data, and no deployed model, and its main finding is negative. The ethical relevance is therefore cautionary. Image classification and semantic segmentation are used in autonomous driving, medical imaging, surveillance, and remote sensing, where a post-hoc normalization swap that looks acceptable on an aggregate metric could still fail on rare classes, under-represented environments, or dense prediction details. Any use of DyT replacement in a real application should therefore be validated against the original LayerNorm model on the target dataset, including per-class accuracy or per-class IoU rather than only aggregate top-1 or mIoU.

7. Conclusion

Dynamic Tanh was validated as a normalization replacement when present from the start of training. This paper tests the more practical post-hoc setting: replacing LayerNorm inside a pretrained vision encoder and relying on downstream fine-tuning to recover performance. Across ViT-B/16 classification and SegFormer-B0 segmentation, full replacement is not competitive with the LayerNorm baseline. Affine preservation substantially improves classification over DyT default, but it still leaves a large accuracy

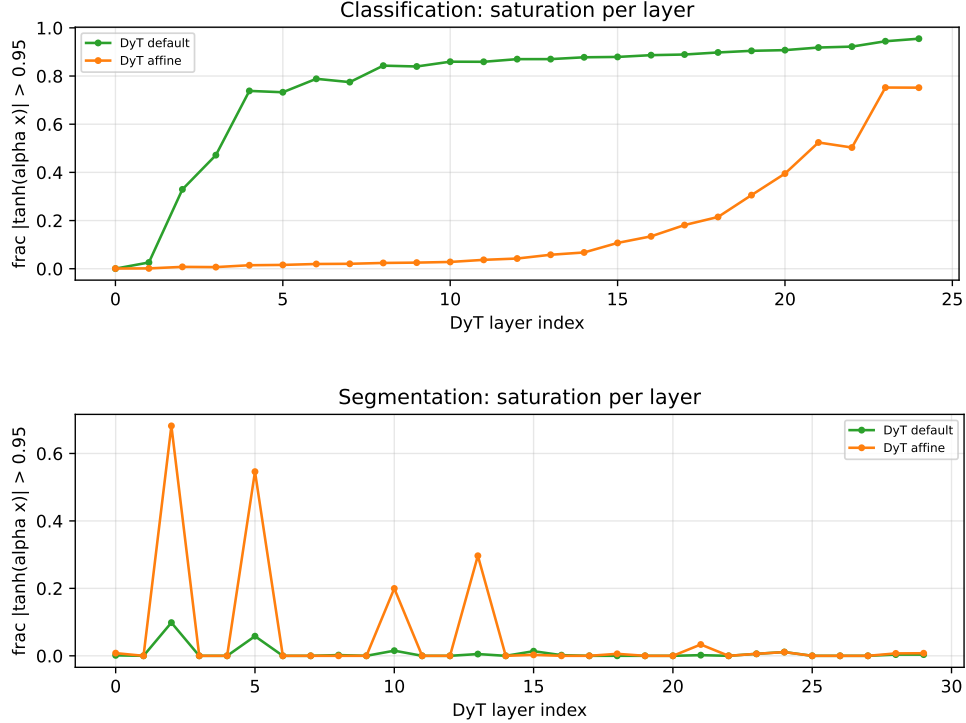


Figure 4. Per-layer DyT saturation fraction, time-averaged over recorded forward passes during training and evaluation as the fraction of activations satisfying $|\tanh(\alpha x)| > 0.95$. Top: classification (ViT-B/16). Bottom: segmentation (SegFormer-B0). High values indicate feature-range collapse; low values indicate the layer operates near the linear regime of $\tanh$.

gap; in segmentation, the same adaptation does not help.

The main contribution is a bounded negative result with diagnostics. Post-hoc DyT failure is not a single phenomenon: ViT DyT default failure is consistent with $\tanh$ saturation, while SegFormer failure occurs despite low saturation and points toward sensitivity to the loss of per-token mean and variance. Follow-up directions, from compute scaling to site-selective replacement and activation-matching initialization, are detailed in Sec. 6.

8. AI Use Statement

Claude Code and Codex were used only to assist with brainstorming methodology, finding relevant literature, and LaTeX formatting tasks; all other work was completed by Kyle Luo without AI assistance.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1290–1299, 2022.
- [3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [4] Stefan Heimersheim. You can remove GPT2’s LayerNorm by fine-tuning. *arXiv preprint arXiv:2409.13710*, 2024.
- [5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.
- [6] Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, and Neil Houlsby. Big transfer (BiT): General visual representation learning. In *European Conference on Computer Vision (ECCV)*, 2020.
- [7] Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [8] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

- [9] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Saško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussière, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, François Lagunas, Alexander M. Rush, and Thomas Wolf. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184. Association for Computational Linguistics, 2021.
- [10] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440, 2015.
- [11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [12] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. <https://github.com/open-mmlab/mms Segmentation>, 2020.
- [13] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [14] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12179–12188, 2021.
- [15] Antonio Torralba, Phillip Isola, and William T. Freeman. *Foundations of Computer Vision*. Adaptive Computation and Machine Learning. MIT Press, 2024.
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [17] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.
- [18] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 12077–12090, 2021.
- [19] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.
- [20] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [21] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 633–641, 2017.
- [22] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14901–14911, 2025.